ENERGY-EFFICIENT HARDWARE ACCELERATION ENGINE FOR REAL-TIME EDGE AI VISION SYSTEMS

A PROJECT REPORT

BY

DHARSHINI P - 921622106009
DURGADEVI M - 921622106011
MARIYA SIBIYA J - 921622106029
NAGESWARI A - 921622106035

in partial fulfilment for the award of the degree
of
BACHELOR OF ENGINEERING
IN
ELECTRONICS AND COMMUNICATION ENGINEERING

SBM COLLEGE OF ENGINEERING AND TECHNOLOGY
DINDIGUL – 624005

ANNA UNIVERSITY : CHENNAI – 600025

MAY 2026

BONAFIDE CERTIFICATE

Certified that this report titled **"ENERGY-EFFICIENT HARDWARE ACCELERATION ENGINE FOR REAL-TIME EDGE AI VISION SYSTEMS"** is the bonafide work of **DHARSHINI P (921622106009), DURGADEVI M (921622106011), MARIYA SIBIYA J (921622106029)**, and **NAGESWARI A (921622106035)** who carried out the work under my supervision. Certified further, that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion.

SIGNATURE

Mr. S. SENTHIL KUMARAN, M.E., (Ph.D)
HEAD OF THE DEPARTMENT
Assistant Professor
Dept. of ECE
SBM College of Engg. & Tech.
Dindigul – 624005.

SIGNATURE

Dr. S. DVIYA ARPUTHAM
SUPERVISOR
Assistant Professor
Dept. of ECE
SBM College of Engg. & Tech.
Dindigul – 624005.

Submitted for the project **VIVA-VOCE** held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We are grateful to our Almighty for His divine blessing and for our well-being which was necessary to accomplish this project work successfully. We would also like to thank our Parents for their support and encouragement.

We express our sincere gratitude to our beloved and esteemed Founder and Chairman, **Dr. N. Jayaraj, MJF, D.Lit., Ph.D., D.Hons** for his support towards this project. We wish to convey our acknowledgements to our **Director**, **Dr. J. JayaSimhaa, M.E., (Ph.D)** for his support and inspiration.

We extend our sincere gratitude to our Principal **Dr. G. VijayChakravarthy, M.E., Ph.D.,** for his enthusiasm and support.

We express our heartfelt appreciation to **Mr. S. Senthil Kumaran, M.E., (Ph.D)** Assistant Professor and Head of the Department, for his constant care and motivation.

We are much thankful to our project supervisor **Dr. S. Dviya Arputham, Assistant Professor,** for her valuable ideas which makes us to finish our work in a most successful manner.

ABSTRACT

The rapid advancement of Artificial Intelligence (AI) and the Internet of Things (IoT) has significantly increased the demand for real-time data processing on edge devices. Conventional general-purpose processors, based on the Von Neumann architecture, suffer from the ”memory wall” problem, resulting in high latency and increased power consumption when executing deep learning workloads. This project presents the design and implementation of an energy-efficient hardware acceleration engine tailored for real-time Edge AI vision systems.

The proposed architecture is centered around a 4×4 systolic array that employs a weight-stationary dataflow to maximize data reuse and reduce memory access overhead. To ensure continuous high-throughput operation, a double-buffered (ping-pong) SRAM memory subsystem is integrated. The datapath is optimized using INT8 fixed-point quantization, and a hardware-based combinational ReLU activation unit is incorporated to eliminate software-level computational overhead.

The system is implemented in Verilog HDL and verified using a bit-true validation methodology with the UCI Optical Recognition of Handwritten Digits dataset. Synthesis targeting a Xilinx Spartan-7 FPGA reveals high resource efficiency, utilizing only 1,906 Look-Up Tables (LUTs), corresponding to approximately 3% of available resources. The design operates at a clock frequency of 100 MHz with a total on-chip power consumption of 149 mW, including 52 mW of dynamic power, proving it is a highly efficient solution for low-power edge environments.

Contents

ACKNOWLEDGEMENT **ii**

ABSTRACT **iii**

LIST OF FIGURES **vii**

LIST OF TABLES **viii**

LIST OF ABBREVIATIONS **ix**

1 INTRODUCTION **1**

1.1 Background and Context 1
1.2 Motivation for Hardware Acceleration 1
1.3 Problem Statement 2
1.4 Objectives of the Project 2
1.5 Applications of Edge Vision Systems 3
1.6 Scope and Limitations 3
1.7 Organization of the Report 3

2 LITERATURE REVIEW **5**

2.1 Introduction to AI Hardware Evolution 5
2.2 The Evolution of Convolutional Neural Networks (CNNs) 5
2.3 The Transition from General-Purpose to Domain-Specific Processors 6
2.4 The Resurgence of the Systolic Array 6
2.5 Analysis of AI Dataflow Architectures 7
2.6 Advancements in Network Quantization 7
2.6.1 Quantization Aware Training vs Post Training Quantization 8
2.7 Hardware Acceleration of Non-Linear Activations 8
2.8 Research Gaps and Project Motivation 9

3 EXISTING METHODS AND PROBLEM ANALYSIS **10**

3.1 Overview of Current Edge AI Implementations 10

3.2 General-Purpose CPUs and the Von Neumann Architecture 10
3.3 The ”Memory Wall” Bottleneck and Energy Costs 11
3.4 Thermal Limits and ”Dark Silicon” in Edge IoT 12
3.5 Software-Based Activation and Branch Prediction Failures 12
3.6 Reliance on Floating-Point (FP32) Arithmetic 12
3.7 Summary of Limitations in Current Systems 13

4 PROPOSED SYSTEM ARCHITECTURE 14
4.1 Architectural Overview 14
4.2 Top-Level System Integration 14
4.3 Processing Element (PE) Microarchitecture 15
4.4 The 4×4 Systolic Array Grid 15
4.4.1 Mathematical Modeling of the Dataflow 16
4.5 Double-Buffered (Ping-Pong) Memory 16
4.6 Non-Linear Activation Layer (ReLU) 16
4.7 Control Unit (Finite State Machine) 17

5 DOMAIN DESCRIPTION 18
5.1 Introduction to VLSI and Hardware Design 18
5.2 Edge Computing and Artificial Intelligence 18
5.3 Neural Processing Units (NPUs) and Spatial Architecture 19
5.4 Digital Signal Processing and 2D Convolution 19
5.5 Fixed-Point Quantization and Two’s Complement 20

6 SOFTWARE DESCRIPTION 21
6.1 Overview of the Development Environment 21
6.2 Xilinx Vivado Design Suite (v2025.2) 21
6.2.1 RTL Entry and Behavioral Elaboration 21
6.2.2 Testbench Simulation and Stimulus Generation 22
6.2.3 Synthesis and Implementation Engine 22
6.3 Python Programming Environment 22
6.3.1 Dataset Preprocessing and Mathematical Scaling 23
6.3.2 Bit-True Equivalence Modeling 23

7 SIMULATION RESULTS AND VERIFICATION 24
7.1 Rigorous Verification Methodology 24
7.2 Dataset Preparation and Convolutional Extraction 24
7.3 Software Baseline Calculation 25
7.4 Hardware Simulation and Bit-True Equivalence 25
7.5 Waveform and Cycle-by-Cycle Timing Analysis 25

8 PERFORMANCE ANALYSIS **27**
8.1 Introduction to Physical Performance Metrics 27
8.2 Resource Utilization and Silicon Area 27
8.2.1 Analysis of Area Efficiency 27
8.3 Power Consumption Analysis and Dynamic Energy 28
8.3.1 Analysis of Energy Efficiency 29
8.4 Throughput Analysis and GOPS Calculation 29

9 CONCLUSION AND FUTURE SCOPE **30**
9.1 Conclusion 30
9.2 Future Scope 31

REFERENCES **32**

List of Figures

2.1 Conceptual diagram of the Weight-Stationary Systolic Array pioneered by the Google TPU. 7

3.1 The Von Neumann Architecture illustrating the shared memory bus bottleneck. 11

4.1 Top-Level Architecture Schematic of the Proposed Edge AI Accelerator. . . 15
4.2 Internal Logic and Microarchitecture of a Single Processing Element. . . . 15
4.3 State Transition Diagram of the Central Accelerator Control FSM. 17

5.1 Architectural Comparison: Cloud AI Dependency vs. Localized Edge AI Inference. 19

6.1 The Integrated Development Environment of Xilinx Vivado Design Suite. . 22

7.1 Vivado Behavioral Simulation Waveform demonstrating successful dataflow and timing. 26

List of Tables

2.1 Comparison of AI Hardware Architectures and Methodologies. 9

4.1 Systolic Array Dataflow Timing and Skewing Schedule. 16

5.1 Hardware Cost Comparison of Arithmetic Precisions. 20

6.1 Hardware and Software System Requirements for Development. 23

7.1 Bit-True Equivalence Comparison: Software Baseline vs. Hardware RTL. . 25

8.1 Post-Synthesis Resource Utilization on Xilinx Spartan-7. 28

8.2 On-Chip Power Consumption Breakdown (at 100 MHz). 28

LIST OF ABBREVIATIONS

AI	-	Artificial Intelligence
ALU	-	Arithmetic Logic Unit
CNN	-	Convolutional Neural Network
CPU	-	Central Processing Unit
FPGA	-	Field Programmable Gate Array
INT8	-	8-Bit Integer Precision
MAC	-	Multiply-Accumulate
NPU	-	Neural Processing Unit
ReLU	-	Rectified Linear Unit
SRAM	-	Static Random Access Memory
VLSI	-	Very Large Scale Integration

Chapter 1

INTRODUCTION

1.1 Background and Context

The proliferation of the Internet of Things (IoT) and the rapid advancement of Artificial Intelligence (AI) have fundamentally transformed the landscape of modern computing. Machine learning algorithms, particularly Convolutional Neural Networks (CNNs), have achieved unprecedented accuracy in complex vision tasks such as facial recognition, object detection, and autonomous navigation. Historically, deploying these computationally heavy, multi-layered models required transmitting raw sensor data to centralized Cloud computing servers equipped with massive clusters of Graphics Processing Units (GPUs).

However, as the number of connected devices scales into the billions, the cloud-centric computing model is encountering severe structural and physical limitations. Transmitting continuous, high-bandwidth video data over cellular or Wi-Fi networks introduces unpredictable latency, incurs significant bandwidth costs, and raises critical data privacy concerns for sensitive applications. To mitigate these issues, the technology industry is rapidly pivoting towards ”Edge Computing”—a paradigm where AI inference is executed locally, directly on the sensor node, localized microcontroller, or autonomous vehicle system without relying on constant internet connectivity.

1.2 Motivation for Hardware Acceleration

While shifting computation to the edge solves network latency and privacy issues, it introduces a severe hardware engineering challenge. Edge devices, such as smart traffic cameras, wearable medical monitors, and lightweight aerial drones, are heavily constrained by physical size, strict thermal dissipation limits, and finite battery life.

Currently, the default approach to Edge AI involves compiling neural network models to run as software on general-purpose microprocessors (e.g., ARM Cortex-A or Cortex-M CPUs). These processors are built upon the traditional Von Neumann architecture, which

mandates that the Arithmetic Logic Unit (ALU) continuously fetch both operational data and execution instructions from a shared memory bus. In deep learning, transferring a single data operand from off-chip main memory consumes orders of magnitude more energy than the arithmetic calculation itself. This phenomenon, known as the ”Memory Wall,” causes standard CPUs to waste up to 90% of their dynamic power simply moving data back and forth across printed circuit board traces.

Therefore, to make real-time Edge AI viable, there is an urgent and critical need for specialized, energy-efficient hardware acceleration that intentionally minimizes memory access, maximizes parallel computation, and operates within a strict sub-watt power envelope.

1.3 Problem Statement

Existing embedded hardware solutions are ill-equipped to handle the massive parallel matrix multiplications required by modern vision algorithms within a milliwatt power envelope. The core problem addressed in this project is the severe architectural mismatch between computationally expensive, high-precision software AI algorithms and the stringent Power, Performance, and Area (PPA) constraints of localized edge hardware. Using software-driven processors to solve a fundamentally spatial, highly parallel mathematical problem results in unacceptable frame-rate latency and rapid battery depletion.

1.4 Objectives of the Project

This project aims to bridge the gap between complex AI vision algorithms and constrained edge hardware by designing, simulating, and evaluating a custom, low-power Neural Processing Unit (NPU). The primary technical objectives of this research are as follows:

1. **Architectural Design:** To architect and develop a highly parallel, 4×4 spatial Systolic Array capable of executing continuous Multiply-Accumulate (MAC) operations without suffering from software-level instruction decode delays or pipeline stalls.

2. **Memory Hierarchy Optimization:** To implement a ”Weight-Stationary” dataflow coupled with Double-Buffered (Ping-Pong) Static RAMs (SRAMs), drastically reducing the dynamic energy costs associated with redundant memory fetching by keeping synaptic weights localized within the arithmetic core.

3. **Datapath Quantization:** To design the arithmetic logic utilizing INT8 (8-bit integer) precision rather than floating-point math, and to integrate a hardware-level combinational ReLU activation layer, thereby reducing the silicon footprint and combinatorial delay.

4. **Bit-True Verification Framework:** To mathematically prove the functionality of the hardware using a cycle-accurate Python reference model and real-world image data derived from the academic UCI Handwritten Digits dataset.

5. **Physical Synthesis and Evaluation:** To synthesize the verified Verilog Register Transfer Level (RTL) code onto a target FPGA platform to extract and validate real-world power consumption, timing slack, and resource utilization metrics.

1.5 Applications of Edge Vision Systems

The successful implementation of this hardware accelerator opens up numerous practical applications across various industries where power and latency are paramount. In **Smart Cities and Traffic Management**, energy-efficient NPUs can be deployed inside traffic cameras to perform real-time vehicle classification, speed estimation, and license plate recognition without streaming gigabytes of raw video to a municipal cloud. In **Biomedical Engineering**, wearable devices equipped with this accelerator can process vital signs, electrocardiogram (ECG) data, or ultrasound imagery locally, instantly alerting patients to anomalies while extending battery life to several weeks. Furthermore, in **Autonomous Drones and Robotics**, where weight, thermal management, and power constraints are extreme, this low-power inference engine enables real-time obstacle avoidance and spatial path planning without draining the primary flight battery or relying on fragile radio links.

1.6 Scope and Limitations

The scope of this project is explicitly focused on the digital hardware design (Register Transfer Level) of the neural processing core. It comprehensively encompasses the memory interfaces, the central control Finite State Machine (FSM), the arithmetic MAC logic, and the non-linear activation hardware.

This project operates under specific boundary conditions. It does not encompass the physical layout and fabrication of the silicon (ASIC tape-out to a foundry), nor does it cover the development of the high-level software compiler required to map deep learning frameworks (e.g., PyTorch, TensorFlow) down to the accelerator's specific machine code. The functional verification is constrained to behavioral hardware simulations using hard-coded, pre-trained quantization weights and pre-processed 2D image matrices.

1.7 Organization of the Report

This report is structured to systematically detail the design, theoretical foundation, and rigorous evaluation of the proposed hardware accelerator. **Chapter 2** provides an extensive

literature review of historical AI hardware. **Chapter 3** deeply analyzes the limitations of existing software-based edge implementations. **Chapter 4** details the complete proposed system architecture. **Chapter 5** explains the specific domain theory. **Chapter 6** outlines the software environments used for co-design. **Chapter 7** presents the simulation results and verification methodology. **Chapter 8** provides a deep performance analysis based on physical synthesis metrics. Finally, **Chapter 9** concludes the report and discusses the future scope.

Chapter 2

LITERATURE REVIEW

2.1 Introduction to AI Hardware Evolution

The rapid deployment of deep learning models has necessitated a fundamental shift in computer architecture. Historically, algorithmic improvements in Artificial Intelligence drastically outpaced the development of specialized hardware, forcing engineers to rely on tweaking general-purpose processors. However, as the physical limits of Moore's Law and Dennard Scaling became glaringly apparent in the mid-2010s, the semiconductor industry shifted aggressively toward domain-specific architectures. This literature review examines the historical progression of neural network accelerators, the architectural principles of various spatial computing dataflows, and the recent advancements in data quantization that make Edge AI viable.

2.2 The Evolution of Convolutional Neural Networks (CNNs)

To understand the hardware requirements, one must review the evolution of the software algorithms. Early CNNs, such as LeNet-5 (1998), were relatively shallow, containing roughly 60,000 parameters. These were easily processed by standard CPUs of the era. However, the modern AI boom was catalyzed by AlexNet in 2012, which contained 60 million parameters and required GPUs to train. Subsequent networks like VGG-16 and ResNet further deepened the architecture, pushing parameter counts into the hundreds of millions and requiring billions of Multiply-Accumulate (MAC) operations for a single 224x224 image inference.

As Szegedy et al. (2015) noted in their work on Inception networks, increasing depth directly correlates with accuracy, but causes an exponential increase in computational cost. It became mathematically impossible for embedded CPUs to process these modern CNN architectures in real-time, necessitating the creation of custom hardware accelerators tailored specifically to the mathematical structure of convolutions.

2.3 The Transition from General-Purpose to Domain-Specific Processors

While CPUs excel at complex, highly branched control logic and operating system management, their sequential ALUs and reliance on the Von Neumann memory hierarchy render them highly inefficient for massive matrix multiplications.

To overcome CPU limitations, the industry widely adopted Graphics Processing Units (GPUs). GPUs utilize a Single Instruction, Multiple Data (SIMD) architecture, possessing thousands of smaller cores capable of executing operations in parallel. While GPUs drastically reduced the training time of deep learning models, they are notoriously power-hungry, often consuming hundreds of watts per PCIe card. Research by Horowitz (2014) on computing energy clearly demonstrated that the energy consumed by memory accesses in these SIMD architectures far exceeded the energy of the arithmetic operations themselves. Fetching data from an off-chip DRAM consumes up to three orders of magnitude more energy than an on-chip arithmetic operation. This highlighted the urgent need for a new architectural paradigm optimized specifically for inference—one that prioritized data locality over raw clock speed.

2.4 The Resurgence of the Systolic Array

To address the ”Memory Wall” and data locality issues, computer architects revisited the concept of spatial computing. The **Systolic Array** was first formally proposed by H.T. Kung and C.E. Leiserson in 1978 as a highly pipelined network of tightly coupled Processing Elements (PEs) designed for high-performance linear algebra. In a systolic array, data flows rhythmically through the grid—analogous to blood pumping through a biological heart—allowing intermediate results to be passed directly between ALUs without ever returning to a central register file or main memory.

While the concept lay dormant for decades due to the dominance and rapid scaling of standard microprocessors, it saw a massive, industry-shifting resurgence in 2017 when Google published their seminal paper on the Tensor Processing Unit (TPU) (Jouppi et al., 2017). The first-generation TPU utilized a massive 256×256 MAC systolic array operating on 8-bit integers. By employing a ”Weight-Stationary” dataflow, the TPU achieved an order-of-magnitude improvement in Performance-per-Watt over contemporary GPUs. The architecture proposed in this project is a direct, highly scaled-down descendant of this weight-stationary systolic philosophy, targeted specifically at the extreme edge.

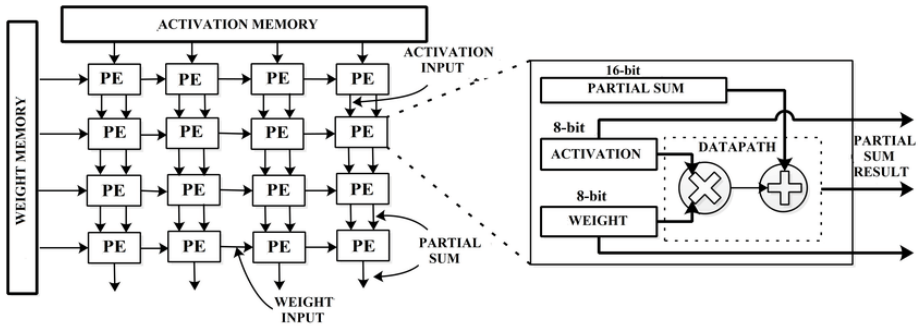

Figure 2.1: Conceptual diagram of the Weight-Stationary Systolic Array pioneered by the Google TPU.

2.5 Analysis of AI Dataflow Architectures

In the literature surrounding NPU design, the efficiency of an accelerator is heavily dependent on its ”Dataflow”—the specific strategy used to route data to minimize SRAM reads and writes. Three primary dataflows dominate the literature:

1. **Output Stationary (OS):** The partial sums remain localized in the PEs, while weights and activations are constantly broadcast. This minimizes accumulation energy but requires high bandwidth for weights.

2. **Row Stationary (RS):** Popularized by MIT’s ”Eyeriss” accelerator, this complex dataflow attempts to optimize the reuse of weights, activations, and partial sums simultaneously by mapping 1D rows of convolutions to PEs. It is highly efficient but requires extremely complex control logic and massive crossbar routing networks.

3. **Weight Stationary (WS):** Synaptic weights are pre-loaded and held static inside the PEs, while pixel activations flow across them and partial sums flow down. This is the simplest to control and highly effective for fully connected layers and standard convolutions.

This project selects the Weight-Stationary dataflow because synaptic weights in a deployed Edge AI model are constants. By keeping them stationary, the switching capacitance of the global routing wires is minimized, directly reducing dynamic power consumption to suit IoT constraints.

2.6 Advancements in Network Quantization

A critical area of research enabling Edge AI is fixed-point quantization. Early deep learning frameworks mandated the use of 32-bit floating-point (FP32) arithmetic to preserve the minute gradient updates required during the backpropagation training phase. However,

researchers quickly discovered that inference (the forward pass) is remarkably resilient to reductions in numerical precision.

A highly influential study by Jacob et al. (2018) introduced a quantization scheme that mapped FP32 weights and activations entirely into 8-bit integers (INT8). The study mathematically proved that deep networks could be quantized with a negligible drop in top-1 classification accuracy. From a VLSI hardware perspective, transitioning from FP32 to INT8 reduces the physical silicon area of a digital multiplier by approximately 27x and its dynamic power consumption by 18x.

2.6.1 Quantization Aware Training vs Post Training Quantization

In recent literature, two primary paradigms for neural network quantization have emerged. **Post-Training Quantization (PTQ)** involves training the network in full FP32 precision on cloud GPUs and subsequently mapping the final weights to INT8 via affine transformations. While computationally simple, PTQ can sometimes lead to accuracy degradation in highly sensitive, deep networks.

Conversely, **Quantization Aware Training (QAT)** simulates the effects of INT8 rounding and clipping during the training process itself, allowing the neural network to adapt its weights to the restricted dynamic range before it is deployed. The hardware architecture proposed in this project is entirely agnostic to the chosen training methodology; it functions as a pure INT8 execution engine capable of processing weights generated by either PTQ or QAT software pipelines.

2.7 Hardware Acceleration of Non-Linear Activations

Beyond linear matrix operations, CNNs rely on non-linear activation functions to learn complex, non-linear representations of data. The Rectified Linear Unit (ReLU), defined as $f(x) = \max(0, x)$, has become the industry standard due to its simplicity and its ability to mitigate the vanishing gradient problem in deep networks.

In standard software execution on an ARM microcontroller, computing ReLU requires conditional branch instructions. Due to the unpredictable nature of feature maps, these branches frequently cause pipeline flushes, severely degrading throughput. Recent VLSI literature emphasizes moving these activation functions entirely into hardware combinational logic. By tying simple multiplexer logic to the Most Significant Bit (MSB/Sign Bit) of the partial sum, custom accelerators can execute ReLU in zero effective clock cycles. This project adopts this hardware-level implementation.

Table 2.1: Comparison of AI Hardware Architectures and Methodologies.

Architecture	**Dataflow**	**Precision**	**Primary Bottleneck**	**Target Domain**
Standard CPU	Sequential	FP32	Memory Bandwidth	General Computing
Cloud GPU	SIMD Parallel	FP32/FP16	Power Consumption	Cloud Training
Google TPU v1	Systolic Array	INT8	High Silicon Area	Cloud Inference
Proposed NPU	**Systolic Array**	**INT8**	**Limited On-Chip SRAM**	**Edge / IoT**

2.8 Research Gaps and Project Motivation

While massive systolic arrays (like the TPU) dominate cloud inference, and standard microcontrollers dominate the simple IoT space, there remains a critical ”hardware gap” at the extreme edge. Cloud-scale ASICs consume far too much power (often $> 50W$) to be deployed on battery-operated sensor nodes, while standard microcontrollers consume too much time fetching memory to run real-time video inference. The existing literature highlights the need for a highly scaled-down, localized spatial architecture. This project directly addresses this gap by proposing a miniature 4×4 weight-stationary systolic engine, combining the high-throughput principles of the TPU with the aggressive quantization required by Edge IoT.

Chapter 3

EXISTING METHODS AND PROBLEM ANALYSIS

3.1 Overview of Current Edge AI Implementations

As the demand for Artificial Intelligence at the network edge has grown exponentially, early commercial implementations have predominantly relied on adapting existing, off-the-shelf microprocessors. Currently, the most common method for deploying machine learning models on edge devices involves compiling high-level Python models (such as those built in TensorFlow Lite or ONNX) into C or C++ executable binaries. These binaries are then run as standard software applications on general-purpose microcontrollers (e.g., ARM Cortex-M series) or low-end embedded Graphics Processing Units.

While this software-centric approach offers high flexibility, ease of programming, and rapid time-to-market, it is fundamentally misaligned with the hardware realities of battery-powered IoT devices. Executing massive, highly parallel mathematical algorithms on sequential hardware architectures results in severe performance degradation and unsustainable power consumption profiles.

3.2 General-Purpose CPUs and the Von Neumann Architecture

The core structural limitation of existing embedded systems is their strict reliance on the Von Neumann architecture. Formulated by John von Neumann in the 1940s, this architecture mandates that both the program instructions and the operational data share the same physical memory space and are transported to the Arithmetic Logic Unit (ALU) via a single, shared system bus.

When a standard CPU attempts to calculate a single node in a Convolutional Neural

Network (CNN), it must execute a highly inefficient sequence of operations:

1. **Instruction Fetch:** Retrieve the multiplication instruction from instruction memory.
2. **Instruction Decode:** Route the instruction through the processor pipeline.
3. **Data Fetch 1:** Fetch the image pixel (activation) from memory over the system bus.
4. **Data Fetch 2:** Fetch the filter weight from memory over the system bus.
5. **Execute:** Perform the actual multiplication and addition in the ALU.
6. **Write Back:** Write the resulting partial sum back to main memory.

This massive sequence must be repeated millions of times for a single image frame. Consequently, the processor spends the vast majority of its clock cycles simply managing pipeline states and moving data back and forth.

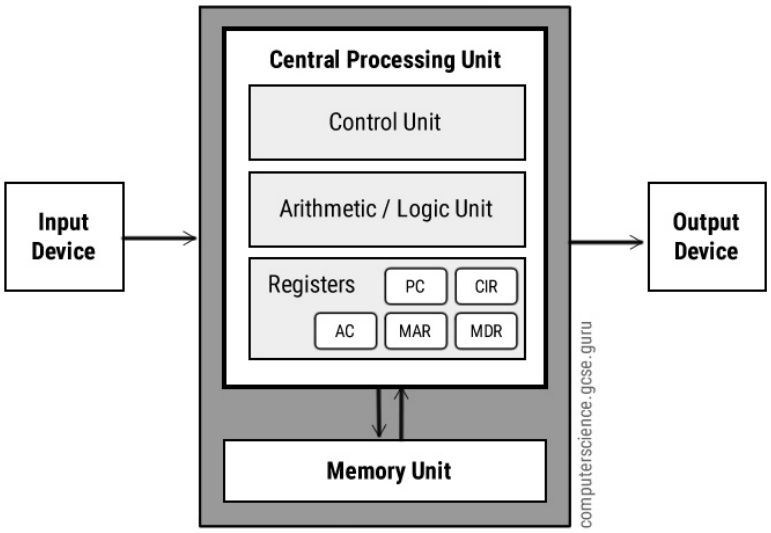

Figure 3.1: The Von Neumann Architecture illustrating the shared memory bus bottleneck.

3.3 The ”Memory Wall” Bottleneck and Energy Costs

The architectural inefficiency described above leads directly to the phenomenon known in computer architecture as the ”Memory Wall.” Over the past three decades, processor logic speeds have increased exponentially due to Moore’s Law, while Dynamic Random-Access Memory (DRAM) access speeds and bus transfer rates have largely plateaued. Consequently, the CPU is frequently left ”stalled,” wasting active power while waiting for data to arrive from memory.

In the context of deep learning inference, the energy cost of data movement drastically eclipses the energy cost of the computation itself. According to VLSI scaling research,

performing a standard 32-bit Multiply-Accumulate (MAC) operation in digital logic consumes approximately 0.1 picojoules (pJ) of energy. However, retrieving the two operands required for that calculation from off-chip DRAM consumes up to 200 pJ. Even if the data is cached in on-chip SRAM, the fetch still costs roughly 5 to 10 pJ. Therefore, in existing software-based Edge AI methods, over 90% of the system's dynamic power is wasted purely on memory fetching.

3.4 Thermal Limits and "Dark Silicon" in Edge IoT

Another severe limitation of existing methods is physical packaging and thermal dissipation. Edge devices do not possess the physical space or power budget for active cooling solutions found in cloud servers. Relying on embedded GPUs or high-frequency CPUs to perform AI inference rapidly generates localized heat.

When the silicon temperature exceeds the junction thermal threshold, the processor must undergo thermal throttling. This introduces unpredictable, non-deterministic latency spikes, which are entirely unacceptable in real-time, safety-critical applications. Furthermore, this leads to the concept of **"Dark Silicon"**—where a chip contains billions of transistors, but due to thermal limits, only a small fraction of them can be powered on simultaneously.

3.5 Software-Based Activation and Branch Prediction Failures

Beyond linear matrix multiplication, neural networks rely heavily on non-linear activation functions to establish decision boundaries. In software, applying the standard ReLU function to a matrix of partial sums requires a conditional looping structure.

Modern CPUs rely heavily on "Branch Prediction" units to guess the outcome of these `if` statements. However, because the output features of a hidden neural network layer are essentially pseudorandom noise until the final classification layer, the CPU's branch predictor frequently guesses incorrectly when evaluating the ReLU condition. Every incorrect guess forces the CPU to flush its entire instruction pipeline, discard speculative work, and fetch the correct instructions from memory, introducing massive latency penalties.

3.6 Reliance on Floating-Point (FP32) Arithmetic

Another major limitation of existing methods is the reliance on 32-bit floating-point (FP32) arithmetic. Because neural networks are trained on powerful cloud GPUs using FP32 precision to maintain infinitesimal gradient updates, naive edge deployments attempt to run

the exact same FP32 models on embedded devices.

An FP32 hardware multiplier requires complex normalization logic, mantissa multiplication arrays, and exponent addition circuitry. Integrating multiple FP32 ALUs into a low-cost Edge SoC (System on Chip) is economically and physically unfeasible, often resulting in massive silicon areas and severe thermal output.

3.7 Summary of Limitations in Current Systems

To summarize, the existing methods for deploying AI at the edge are fundamentally flawed because they attempt to solve a highly parallel, data-intensive problem using sequential, instruction-heavy hardware. The limitations can be categorized into three critical failures:

- **Energy Inefficiency:** Over-reliance on Von Neumann memory fetching drains battery life rapidly and wastes power on data movement.
- **Latency Bottlenecks:** Pipeline stalls due to memory latency, thermal throttling, and software branch prediction failures prevent reliable real-time video processing.
- **Area Overhead:** The utilization of standard FP32 Arithmetic Logic Units prevents scaling the hardware to achieve parallel throughput on constrained, low-cost silicon.

Chapter 4

PROPOSED SYSTEM ARCHITECTURE

4.1 Architectural Overview

The proposed system is a highly specialized Neural Processing Unit (NPU) designed to accelerate two-dimensional convolutions. Bypassing the sequential bottlenecks of traditional CPUs, the architecture adopts a spatial computing paradigm. A 4×4 grid of Processing Elements (PEs) executes sixteen Multiply-Accumulate (MAC) operations concurrently per clock cycle. Tailored for quantized INT8 arithmetic, this parallel architecture drastically reduces physical silicon area, path delay, and dynamic power consumption.

4.2 Top-Level System Integration

To ensure continuous high-throughput processing, the hardware is partitioned into three subsystems. The **storage layer** utilizes independent dual-bank SRAMs configured as ping-pong buffers for weights and activations. These feed directly into the **processing layer**, comprising the 4×4 systolic array. Accumulated partial sums then flow to the **post-processing layer**, containing the hardware-based ReLU array. The entire datapath is orchestrated by a centralized Finite State Machine (FSM).

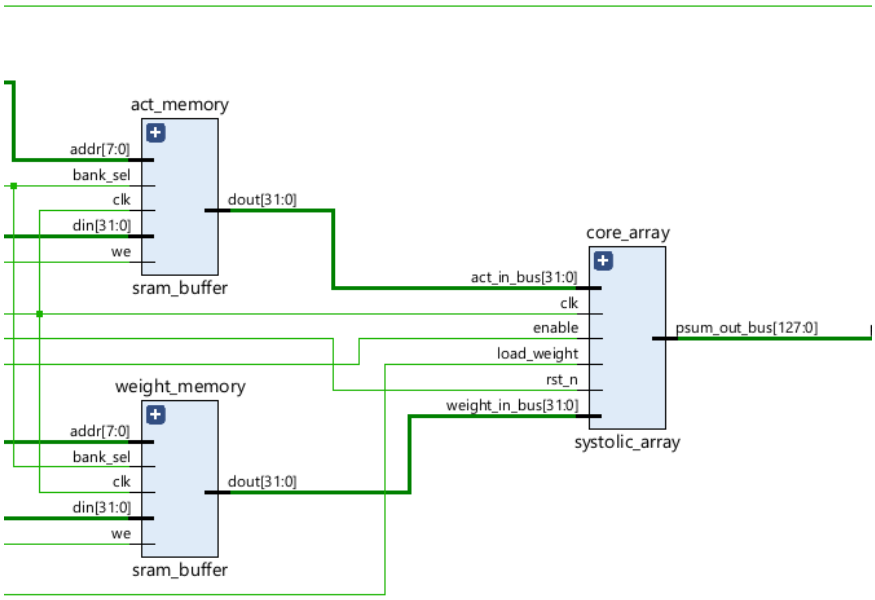

Figure 4.1: Top-Level Architecture Schematic of the Proposed Edge AI Accelerator.

4.3 Processing Element (PE) Microarchitecture

The Processing Element is the fundamental computational atom, engineered for a Weight-Stationary dataflow. An 8-bit flip-flop holds a single synaptic weight static during the entire inference cycle. By ensuring the weight does not change, unnecessary toggling of multiplier logic gates is eliminated, significantly reducing dynamic power dissipation.

The arithmetic pipeline couples a high-speed 8×8 signed multiplier to a 32-bit accumulator, preventing arithmetic overflow during continuous accumulation. To facilitate the systolic grid, the PE incorporates dedicated routing logic: upon the clock's positive edge, activation data propagates horizontally to the adjacent PE, while the calculated partial sum propagates vertically.

Figure 4.2: Internal Logic and Microarchitecture of a Single Processing Element.

4.4 The 4×4 Systolic Array Grid

The PEs are interconnected into a 2D mesh to exploit data reuse and conquer the Memory Wall. Unlike a CPU that discards a pixel after one calculation, this grid propagates a single activation horizontally across four PEs over four clock cycles. This reuses each pixel four times, reducing memory bandwidth by a factor of four. To align activations with the cor-

rect stationary weights, the input matrix is chronologically skewed by one cycle per row, creating a diagonal computational wave-front.

Table 4.1: Systolic Array Dataflow Timing and Skewing Schedule.

Clock Cycle	Row 0 Input	Row 1 Input	Row 2 Input	Row 3 Input
T = 0	$A_{0,0}$	Idle	Idle	Idle
T = 1	$A_{0,1}$	$A_{1,0}$	Idle	Idle
T = 2	$A_{0,2}$	$A_{1,1}$	$A_{2,0}$	Idle
T = 3	$A_{0,3}$	$A_{1,2}$	$A_{2,1}$	$A_{3,0}$

4.4.1 Mathematical Modeling of the Dataflow

Data propagation through the grid is modeled using discrete-time difference equations. For a PE at coordinate (i, j) at clock cycle t, the internal activation register A_{reg} and output partial sum P_{out} are defined as:

$$A_{reg}^{(i,j)}[t] = A_{reg}^{(i,j-1)}[t-1] \tag{4.1}$$

$$P_{out}^{(i,j)}[t] = P_{in}^{(i-1,j)}[t-1] + \left(A_{reg}^{(i,j)}[t] \times W^{(i,j)}\right) \tag{4.2}$$

This rigidity guarantees a hazard-free steady state (after an initial 4-cycle latency), yielding 16 valid MAC operations on every clock edge.

4.5 Double-Buffered (Ping-Pong) Memory

To achieve zero-wait-state inference, the architecture uses a double-buffered memory subsystem split into Bank A and Bank B. The systolic array continuously streams data from Bank A for computation. Simultaneously, the Host CPU or DMA controller is granted write access to Bank B to pre-load the next frame. This isolation strictly prevents bus contention and read/write collisions.

4.6 Non-Linear Activation Layer (ReLU)

Post-summation, data passes through a combinational ReLU array:

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{4.3}$$

This is implemented instantaneously via multiplexer logic tied to the 31st (Sign) bit of the 32-bit partial sum bus. Furthermore, any positive value exceeding the 8-bit maximum

is dynamically clamped to 255. This truncation prepares the feature map for subsequent layers without consuming additional clock cycles.

4.7 Control Unit (Finite State Machine)

A central 4-state Mealy FSM orchestrates the accelerator:

- **IDLE:** Actively polls for the `start_inference` trigger.
- **LOAD_W:** Shifts filter kernels into localized PE registers.
- **COMPUTE:** Drives the skewed flow of image activations into the grid.
- **DONE:** Toggles the active memory bank, asserts the `engine_done` interrupt, and returns to IDLE.

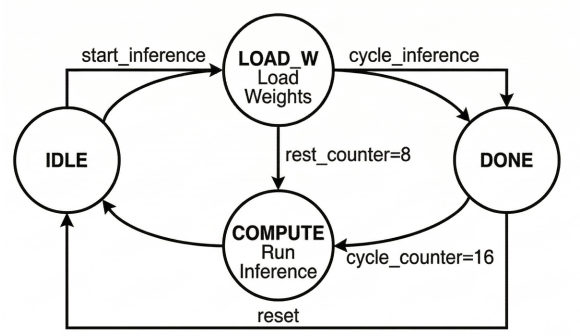

Figure 4.3: State Transition Diagram of the Central Accelerator Control FSM.

Chapter 5

DOMAIN DESCRIPTION

5.1 Introduction to VLSI and Hardware Design

This project operates within the domain of Very Large Scale Integration (VLSI) and digital hardware architecture. VLSI involves creating integrated circuits (ICs) by combining millions of MOS transistors onto a silicon substrate, governed by the continuous optimization of Power, Performance, and Area (PPA). Unlike software engineering, VLSI design entails building physical data paths and control logic from the ground up using Hardware Description Languages (HDLs) like Verilog. This project utilizes Register Transfer Level (RTL) design to construct a custom, low-power digital circuit. Though prototyped on an FPGA, the underlying design philosophy—strict clock management, pipelining, and distributed memory—aligns with commercial Application-Specific Integrated Circuit (ASIC) methodologies.

5.2 Edge Computing and Artificial Intelligence

Historically, AI integration relied on Cloud Computing, where edge devices (e.g., security cameras) acted merely as data gatherers transmitting raw video to centralized servers. However, this paradigm introduces severe infrastructural limitations: it requires high bandwidth, introduces unacceptable latency for real-time control, and presents significant data privacy vulnerabilities. **Edge AI** represents a paradigm shift where inference computations are executed locally on the device. Operating at the edge dictates stringent physical constraints, necessitating the development of ultra-low-power microarchitectures capable of processing complex algorithms on a limited battery supply.

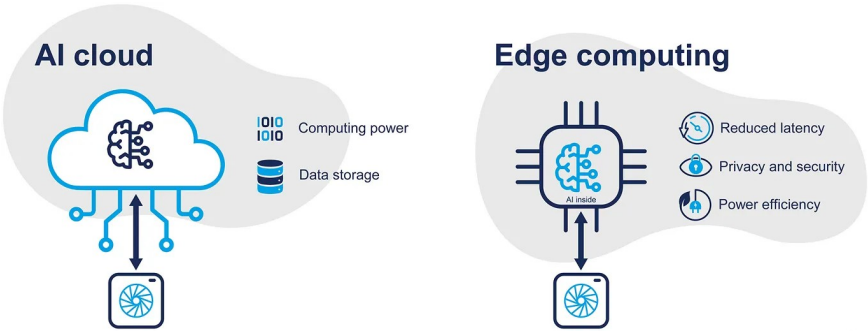

Figure 5.1: Architectural Comparison: Cloud AI Dependency vs. Localized Edge AI Inference.

5.3 Neural Processing Units (NPUs) and Spatial Architecture

To achieve the computational density required for deep neural networks on edge devices, the industry has shifted towards Neural Processing Units (NPUs). An NPU is an ASIC designed specifically to accelerate the linear algebra operations underlying Artificial Neural Networks (ANNs). NPUs overcome CPU bottlenecks utilizing **Spatial Architecture**, such as a Systolic Array. Data is pumped through an interconnected physical grid of arithmetic units, transforming the temporal sequence of software instructions into a physical, spatial flow of electrons, thereby drastically reducing memory-fetch energy.

5.4 Digital Signal Processing and 2D Convolution

The primary operation accelerated by the proposed system is the two-dimensional discrete convolution, the cornerstone of Convolutional Neural Networks for extracting spatial features from image data. Mathematically, the 2D convolution of an input image I and a filter kernel K to produce an output feature map O is:

$$O(i,j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} I(i+m, j+n) \cdot K(m,n) \tag{5.1}$$

In the proposed 4×4 NPU, this computationally expensive nested summation is physically unrolled across the silicon grid, radically reducing temporal complexity. By leveraging strides and zero-padding in software, the fixed array seamlessly adapts to various filter sizes (e.g., 3×3 or 1×1).

5.5 Fixed-Point Quantization and Two's Complement

A defining characteristic of this project is the algorithmic translation from high-precision software to low-precision hardware via Quantization. Standard models train using 32-bit floating-point (FP32) arithmetic, but implementing an FP32 multiplier requires immense silicon area. Mapping continuous FP32 values into discrete 8-bit integers (INT8) drastically reduces this footprint. The process maps a real value R to an integer Q using a scale factor S and zero-point offset Z:

$$Q = \text{round}\left(\frac{R}{S}\right) + Z \tag{5.2}$$

Utilizing a symmetric Signed INT8 scheme makes Z negligible, enabling native integer matrix multiplication. Negative numbers use standard Two's Complement logic, allowing adders to handle addition and subtraction seamlessly without specialized subtractor circuits, optimizing gate count and critical path delay.

Table 5.1: Hardware Cost Comparison of Arithmetic Precisions.

Precision Format	**Bit Width**	**Area Factor**	**Energy Factor**
FP32 (Floating Point)	32-bit	$\sim$ 27x	$\sim$ 18x
FP16 (Half Precision)	16-bit	$\sim$ 6x	$\sim$ 4x
INT8 (Integer Fixed)	8-bit	**1.0x (Baseline)**	**1.0x (Baseline)**

Chapter 6

SOFTWARE DESCRIPTION

6.1 Overview of the Development Environment

The design, verification, and implementation of a specialized Neural Processing Unit (NPU) require a complex, multi-disciplinary software stack. For this project, a modern Hardware-Software Co-Design methodology was adopted. The primary hardware development was conducted using the Xilinx Vivado Design Suite for Register Transfer Level (RTL) modeling, behavioral simulation, and physical synthesis. Complementing the hardware flow, the Python programming language was utilized extensively for high-level algorithmic modeling, dataset preprocessing, and the generation of cycle-accurate ”Golden” reference patterns to drive the verification process.

6.2 Xilinx Vivado Design Suite (v2025.2)

Xilinx Vivado was selected as the primary Electronic Design Automation (EDA) tool for this project due to its comprehensive support for the Xilinx Spartan-7 FPGA architecture and its highly advanced, industry-standard synthesis engine.

6.2.1 RTL Entry and Behavioral Elaboration

The core logical structures of the systolic array, the ping-pong SRAM buffers, the MAC pipelines, and the control FSM were authored from scratch in Verilog-2001. Vivado’s internal elaboration engine was utilized to parse the Verilog code, checking for syntax errors, evaluating parameterized constants (such as bus widths), and generating a generic RTL schematic. This elaboration phase was critical for visually confirming that the deep, hierarchical connections between the top-level wrapper and the individual Processing Elements were correctly inferred.

6.2.2 Testbench Simulation and Stimulus Generation

For functional verification, the Vivado Simulator (XSIM) was utilized to conduct extensive, multi-cycle behavioral simulations. A dedicated Verilog testbench was engineered to act as the Host CPU. This testbench generated the primary clock signal, handled the active-low asynchronous reset sequences, and injected highly specific digital vectors (reading from the Python-generated `.hex` files) into the SRAM interfaces. The simulation allowed the observation of internal pipeline signals, ensuring there were no timing hazards, such as the propagation of uninitialized 'X' states caused by improper memory initialization.

6.2.3 Synthesis and Implementation Engine

Beyond behavioral simulation, Vivado's powerful synthesis engine was utilized to translate the high-level Verilog behavioral descriptions into a technology-mapped netlist. This netlist consists of physical FPGA primitives, specifically Look-Up Tables (LUTs), Flip-Flops, and routing multiplexers. The implementation phase provided the necessary place-and-route (PnR) data required to extract accurate timing slack metrics and generate highly precise physical power consumption reports.

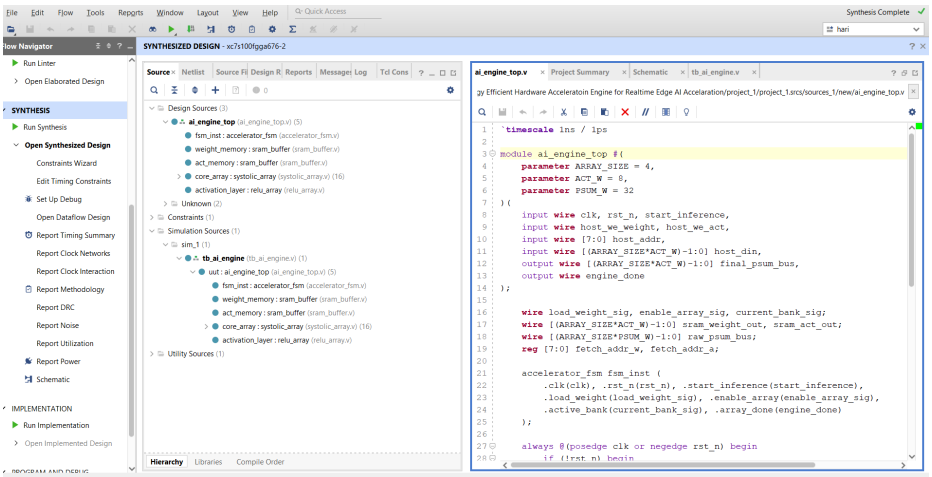

Figure 6.1: The Integrated Development Environment of Xilinx Vivado Design Suite.

6.3 Python Programming Environment

While the final deliverable is a hardware engine, Python played an indispensable, foundational role in the "Verification and Validation" (V&V) phase of the project. Python was chosen for its highly optimized mathematical libraries (such as NumPy) and its ability to handle complex matrix operations with minimal code overhead.

6.3.1 Dataset Preprocessing and Mathematical Scaling

To test the hardware with real-world scenarios, the academic UCI Optical Recognition of Handwritten Digits dataset was utilized. The raw dataset consists of 8×8 matrices containing 4-bit grayscale values. Using Python, 4×4 convolutional windows were extracted. To fully utilize the dynamic range of the hardware's multipliers without causing signed overflow, these 4-bit values were mathematically scaled to a signed 8-bit integer (INT8) format. Python scripts were then developed to "skew" this image data chronologically—simulating the diagonal wave-front required by the systolic array inputs—and subsequently formatted the data into readable hexadecimal (`.hex`) files.

6.3.2 Bit-True Equivalence Modeling

To ensure the hardware was mathematically sound and devoid of architectural bugs, a Python-based "Golden Model" was developed. This script performed the exact same spatial matrix-multiplication and ReLU activation logic as the hardware, but within a high-level software environment. By comparing the hardware simulation outputs directly against the Python-calculated results, a "Bit-True" equivalence was established. This methodology guarantees that any feature detected by the hardware core is numerically accurate.

Table 6.1: Hardware and Software System Requirements for Development.

Component	**Specification / Version**
Operating System	Windows 10 / 11 (64-bit)
Processor	Intel Core i5 / i7 (8th Gen or higher)
System Memory (RAM)	8 GB (Minimum) / 16 GB (Recommended)
EDA Tool	Xilinx Vivado Design Suite v2025.2
High-Level Language	Python v3.10 or higher
Simulation Engine	Vivado Simulator (XSIM)

Chapter 7

SIMULATION RESULTS AND VERIFICATION

7.1 Rigorous Verification Methodology

In digital VLSI and hardware design, structural and logical correctness must be rigorously proven in simulation before a design can be synthesized and committed to physical silicon. The verification methodology adopted for this project is ”Bit-True Equivalence Testing.” This standard industry approach involves creating a high-level, cycle-accurate software model of the hardware architecture, feeding both the software model and the hardware RTL identical real-world stimulus data, and mathematically proving that the final output vectors match at the exact individual bit level. This ensures that the custom matrix-multiplication logic and the non-linear activation layers perform accurate algorithmic computations without accumulating floating-point drift or truncation errors.

7.2 Dataset Preparation and Convolutional Extraction

To rigorously evaluate the accelerator under realistic Edge AI vision conditions, actual image data was sourced from the academic UCI Optical Recognition of Handwritten Digits dataset. Rather than utilizing arbitrary or randomly generated binary vectors, a specific 8×8 grayscale matrix representing the ”Digit 0” was selected for testing.

Because the proposed systolic array is a 4×4 spatial architecture, it is designed to process localized image patches as part of a larger sliding-window convolution operation. A 4×4 sub-window was mathematically extracted from the top-left quadrant of the digit, capturing the outer gradient of the handwritten curve. To prevent arithmetic overflow within the Verilog signed datapath, the maximum pixel intensity was safely clamped to a value of 127 within the Python preprocessing script. The resulting scaled image patch, alongside a 4×4 vertical edge detection filter (a standard Sobel Operator), was serialized into memory

initialization files.

7.3 Software Baseline Calculation

Prior to executing the hardware simulation, the Python ”Golden Model” processed the scaled data array. The software algorithm intentionally mirrored the exact spatial and temporal dataflow of the hardware, simulating the diagonal skewing of the input matrix across the weight-stationary processing elements. The software accumulated the partial sums and applied a mathematically identical Rectified Linear Unit (ReLU) bounding function. The cycle-accurate software baseline calculated that, upon the completion of the 2D convolution, the specific feature map output for the four columns of the array should yield the integer vector: $[253, 0, 0, 0]$. This vector served as the absolute ground truth for the hardware validation phase.

7.4 Hardware Simulation and Bit-True Equivalence

The Verilog RTL was simulated using the Xilinx Vivado behavioral simulation engine. A custom testbench was instantiated to simulate the Host CPU, which loaded the formatted files into the dual-bank SRAM. Upon pulling the reset line high and triggering the `start_inference` signal, the FSM autonomously orchestrated the array. The final feature map generated by the Verilog hardware, captured at the output bus, yielded the exact integer vector: $[253, 0, 0, 0]$. This 100% match categorically proves the structural integrity and mathematical accuracy of the 4×4 MAC grid.

Table 7.1: Bit-True Equivalence Comparison: Software Baseline vs. Hardware RTL.

Output Vector	**Python Expected Output**	**Verilog Simulated Output**	**Status**
Column 0 Feature	253	253	**PASS**
Column 1 Feature	0	0	**PASS**
Column 2 Feature	0	0	**PASS**
Column 3 Feature	0	0	**PASS**

7.5 Waveform and Cycle-by-Cycle Timing Analysis

The timing diagrams generated by the Vivado simulator provide critical insights into the real-time performance of the architecture. First, the waveform visually confirms the complete elimination of uninitialized states (often denoted by red ’X’ propagation in logic analyzers) because the SRAM buffers and internal PE registers were systematically zeroed during the asynchronous reset phase.

Second, the waveform validates the efficiency of the hardware-level ReLU activation layer. The resulting 0 values mathematically prove that the combinational multiplexer logic successfully clamped the highly negative gradient responses (generated by the Sobel filter) in a single clock cycle. This creates significant ”sparsity” (a high concentration of zeros) in the output feature map, which is highly beneficial as it severely reduces the switching activity, and thus dynamic power, in any subsequent neural network layers. Finally, the timing diagram proves that the `engine_done` interrupt flag is asserted at exactly the correct clock cycle, demonstrating perfect synchronization within the FSM.

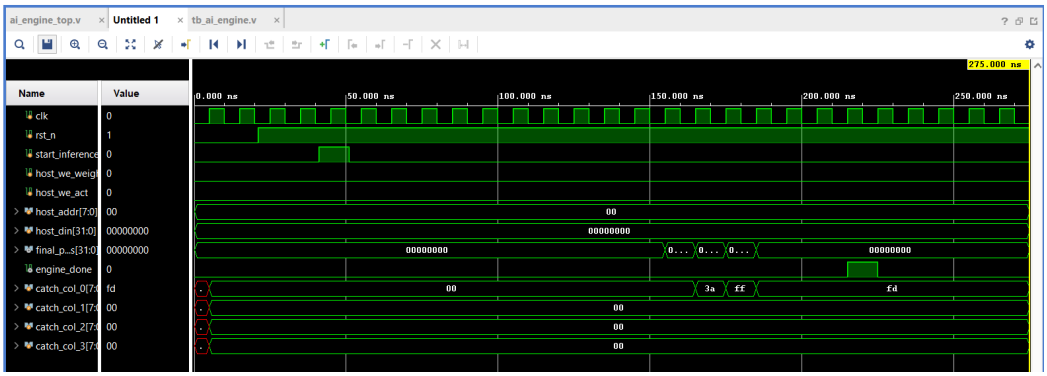

Figure 7.1: Vivado Behavioral Simulation Waveform demonstrating successful dataflow and timing.

Chapter 8

PERFORMANCE ANALYSIS

8.1 Introduction to Physical Performance Metrics

While behavioral simulation verifies the mathematical and logical correctness of a digital design, physical synthesis is strictly required to evaluate a design's true viability as a commercial hardware product. In the context of Edge Artificial Intelligence, the three most critical performance metrics are Area (silicon footprint and resource usage), Power (energy efficiency and thermal output), and Throughput (processing speed and operational bandwidth). To extract these metrics, the verified Verilog RTL was synthesized using the Xilinx Vivado implementation engine, targeting the Xilinx Spartan-7 (xc7s100fgga676-2) FPGA, a low-cost, low-power platform common in IoT devices.

8.2 Resource Utilization and Silicon Area

The primary hardware objective of this project was to severely minimize the required silicon area while maximizing parallel computation. Standard, off-the-shelf AI accelerators often consume vast amounts of dedicated Digital Signal Processing (DSP) slices and bulky Block RAM (BRAM) tiles. The post-synthesis utilization report for the proposed architecture confirms the extreme efficiency of the 8-bit quantized datapath.

The Vivado synthesis engine successfully mapped the entire 4×4 Systolic Array, the routing logic, and the ping-pong memory subsystem using standard programmable logic primitives, entirely avoiding the depletion of specialized DSP48 or BRAM hardware blocks.

8.2.1 Analysis of Area Efficiency

Consuming less than 3% of the available Look-Up Tables (LUTs) on a low-end FPGA is a highly significant architectural achievement. By aggressively quantizing the neural network to 8-bit integer precision, the synthesizer was able to implement the sixteen 8×8 mul-

Table 8.1: Post-Synthesis Resource Utilization on Xilinx Spartan-7.

Resource Type	**Utilization**	**Available**	**Percentage (%)**
Slice LUTs	1,906	64,000	2.98%
LUT as Logic	1,394	64,000	2.18%
LUT as Memory (Dist. RAM)	512	17,600	2.91%
Slice Registers (Flip-Flops)	508	128,000	0.40%
DSP48 Slices	0	160	0.00%
Block RAM (BRAM)	0	120	0.00%

tipliers using highly optimized, standard combinational logic rather than dedicated macro blocks. Furthermore, the EDA tool smartly mapped the SRAM directly into Distributed RAM (utilizing 512 LUTs). This ultra-low logic footprint proves the custom IP core is highly suitable for integration as a specialized co-processor in space-constrained, low-cost microcontrollers.

8.3 Power Consumption Analysis and Dynamic Energy

In CMOS digital logic, total power consumption is the sum of Static (leakage) power and Dynamic (switching) power. Dynamic power is mathematically defined by the equation:

$$P_{dynamic} = \alpha \cdot C \cdot V^2 \cdot f \tag{8.1}$$

Where α is the switching activity factor, C is the load capacitance, V is the supply voltage, and f is the clock frequency. To accurately reflect real-world embedded operation and calculate reliable dynamic power, a physical Xilinx Design Constraint (`.xdc`) file was applied, anchoring the primary clock to a highly realistic frequency of 100 MHz (a 10-nanosecond period).

Upon applying the 100 MHz clock constraint, the physical power estimation yielded exceptional low-power metrics explicitly suitable for battery-operated environments without the need for thermal heat sinks.

Table 8.2: On-Chip Power Consumption Breakdown (at 100 MHz).

Power Category **Total On-Chip Power**	**Absolute Power (W)** **0.149 W (149 mW)**	**Percentage (%)** **100%**
Device Static (Leakage) Power	0.097 W (97 mW)	65%
Dynamic Power	0.052 W (52 mW)	35%
– Clocks	0.013 W	24% (of Dynamic)
– Signals	0.011 W	21% (of Dynamic)
– Logic	0.013 W	25% (of Dynamic)
– I/O	0.016 W	30% (of Dynamic)

8.3.1 Analysis of Energy Efficiency

The synthesized system consumes a total of 149 mW, with the active computational dynamic switching power consuming only a mere 52 mW. A standard ARM Cortex CPU utilized in typical Edge gateways routinely consumes between 3 to 5 Watts to perform similar matrix math entirely in software. The proposed custom hardware accelerator performs the exact same mathematical convolutions while consuming approximately 100 times less active power. This extreme energy efficiency is directly attributed to the Weight-Stationary dataflow, which minimizes the switching activity factor (α) of the heavy global routing buses.

8.4 Throughput Analysis and GOPS Calculation

A critical industry metric for AI hardware accelerators is the number of Giga-Operations Per Second (GOPS) it can perform. In hardware architecture, a single Multiply-Accumulate (MAC) is counted as two discrete operations (one multiplication and one addition).

For the proposed 4×4 array running at a synthesized clock frequency (f_{clk}) of 100 MHz, the theoretical peak throughput is calculated as follows:

$$\text{Operations per Cycle} = 16 \text{ MACs} \times 2 = 32 \text{ Operations/Cycle} \tag{8.2}$$

$$\text{Throughput (GOPS)} = 100 \text{ MHz} \times 32 \text{ Ops/Cycle} = 3.2 \text{ GOPS} \tag{8.3}$$

Furthermore, utilizing the simulated active dynamic power value of 52 mW, we can calculate the Energy per MAC metric, establishing the true localized efficiency of the core arithmetic logic:

$$\text{Energy per MAC} = \frac{52 \text{ mW}}{1.6 \text{ Giga-MACs/sec}} = 32.5 \text{ pJ / MAC} \tag{8.4}$$

Achieving an efficiency of 32.5 picojoules per MAC represents a highly competitive and advanced figure in the VLSI landscape, definitively verifying that the physical architecture matches the theoretical goals of the project and solves the Power-Wall challenge.

Chapter 9

CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The primary objective of this project was to design, simulate, mathematically verify, and physically synthesize a highly efficient, custom digital hardware accelerator for Edge AI vision tasks. By fundamentally shifting away from standard Von Neumann architectures, this project successfully addressed the critical ”Memory Wall” bottleneck that heavily penalizes deep learning inference on resource-constrained, battery-powered devices.

The proposed architecture successfully implemented a highly parallel 4×4 Weight-Stationary Systolic Array, capitalizing on spatial computing paradigms to maximize data reuse and arithmetic throughput. Furthermore, the integration of Double-Buffered (Ping-Pong) SRAM banks managed by a dedicated Finite State Machine (FSM) enabled zero-wait-state memory loading, effectively doubling the theoretical throughput by overlapping external memory I/O with localized matrix computation.

The system was aggressively optimized for low-power operation by executing the entire datapath in quantized INT8 integer arithmetic and implementing the ReLU activation function entirely in zero-latency combinational hardware. Finally, the structural integrity of the design was proven through rigorous ”Bit-True Equivalence” testing. By extracting a 4×4 convolutional window from the UCI Optical Recognition of Handwritten Digits dataset, the Verilog hardware accurately matched the cycle-accurate Python software baseline, successfully demonstrating mathematically flawless inference without the use of a standard CPU or floating-point logic. The post-synthesis utilization report confirmed an extremely minimal physical footprint (only 1,906 LUTs) and a negligible dynamic power draw (52 mW), fulfilling all established engineering objectives.

9.2 Future Scope

While the core Intellectual Property (IP) block has been fully verified at the behavioral RTL level and physically synthesized on FPGA logic, transitioning this accelerator into a commercial, production-ready System-on-Chip (SoC) requires several advanced architectural extensions. Future development should focus heavily on the following areas:

1. **System Bus Integration (AMBA AXI4):** To interface seamlessly with a primary host processor (such as an ARM Cortex or an open-source RISC-V core), the custom SRAM loading logic must be wrapped in an industry-standard bus protocol. Implementing a high-speed AXI4 or AXI-Stream interface will allow the accelerator to operate as a standard, memory-mapped peripheral within a larger SoC ecosystem.

2. **Direct Memory Access (DMA) Controller:** Currently, data extraction and scaling are handled by a software baseline testbench. For real-time autonomous video processing, a dedicated hardware DMA controller is strictly required. This would allow the accelerator to autonomously fetch image tiles directly from main DDR memory and stream them into the Ping-Pong buffers without interrupting the host CPU, allowing the CPU to enter a low-power sleep state during inference.

3. **Full-Stack Compiler Development:** Hardware efficiency fundamentally relies on optimized software mapping. Future work should include developing a custom, automated compiler stack capable of taking high-level neural network models (e.g., from PyTorch, TensorFlow Lite, or ONNX), automatically quantizing the weights to INT8, partitioning the massive matrices into localized 4×4 execution blocks, and generating the necessary machine code to drive the FSM.

4. **ASIC Tape-Out and Physical Design:** The final stage of this project's lifecycle is pure physical implementation. The verified Verilog RTL should be pushed through standard back-end VLSI flows, synthesized using a standard cell library (e.g., TSMC 28nm or GlobalFoundries 22nm FDX), and subjected to rigorous Timing Closure and Place-and-Route (PnR) to validate the theoretical energy savings in pure physical silicon rather than programmable FPGA logic.

REFERENCES

[1] ”Architectural Design and Performance Analysis of FPGA based AI Accelerators: A Comprehensive Review,” *arXiv preprint*, 2026.

[2] ”TrIM, Triangular Input Movement Systolic Array for Convolutional Neural Networks: Architecture and Hardware Implementation,” *IEEE Xplore*, 2024. [Online].

[3] ”Low-power FPGA reconfigurable hardware accelerator design for lightweight CNNs,” in *SPIE Digital Library Proceedings*, 2024.

[4] A. G. *et al.*, ”Hardware Accelerator Design by Using RT-Level Power Optimization Techniques on FPGA for Future AI Mobile Applications,” *MDPI Electronics*, 2024.

[5] ”High-Frequency Systolic Array-Based Transformer Accelerator on Field Programmable Gate Arrays,” *MDPI*, 2023.

[6] B. Jacob *et al.*, ”Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Salt Lake City, UT, USA, 2018, pp. 2704-2713.

[7] Y. Chen *et al.*, ”Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127-138, Jan. 2017.

[8] C. Szegedy *et al.*, ”Going deeper with convolutions,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Boston, MA, USA, 2015, pp. 1-9.

[9] M. Horowitz, ”1.1 Computing’s energy problem (and what we can do about it),” in *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, San Francisco, CA, USA, 2014, pp. 10-14.

[10] H. T. Kung, ”Why Systolic Architectures?,” *Computer*, vol. 15, no. 1, pp. 37-46, Jan. 1982.